

Test Planning Document for TaskAnchor

Version: v1.16

Last Updated: 05/14/2026

1. Purpose

This document defines how the TaskAnchor system is tested to ensure all MVP features function correctly and support continuous task follow-through.

The purpose of testing is to verify that implemented MVP behavior works as expected across backend logic, frontend UI behavior, integration points, browser flows, manual workflows, practical usability of existing MVP screens, and validation/security handling for existing authentication, task, and Progress Log behavior.

MVP feature development is complete. Current testing work is post-MVP validation, documentation alignment, Practical UX Review documentation, Manual MVP Workflow Testing documentation, Playwright coverage for existing MVP flows, demo-readiness confirmation, security review documentation, known-limitation documentation, and future hardening backlog documentation.

Current work log status:

- 70.32 hours worked
- 29.68 hours remaining toward the 100-hour capstone requirement

Remaining time should be used for documentation alignment, demo preparation, presentation/poster preparation, work log completion, final validation checks if needed, bug/limitation documentation, and backlog documentation. Remaining time should not be used to add new MVP features or new workflows.

2. Scope

This testing plan covers MVP features defined in the SRS and validation activities needed for the capstone.

In scope:

- Authentication (register, login)
- Angular UI (Register, Login, Task List, Task Create)
- Task management (create, edit, delete, view)
- Task status transitions
- DueDate and overdue logic
- PriorityLevel
- Task sorting
- Progress Log (add + view)
- NextAction
- Refresh signaling across UI
- Integration testing (UI + API)
- End-to-end testing (Playwright)
- Manual MVP Workflow Testing

- Practical UX Review for existing MVP flows only
- Practical structure/access review for existing MVP screens
- Cosmetic demo-readiness styling for existing MVP screens only
- Existing authentication screen clarity/access corrections
- Existing form validation for required fields
- Existing backend validation hardening for selected task/authentication/progress-log inputs
- Existing authentication input validation and suspicious-input handling tests
- Existing task/progress-log suspicious-input tests
- Existing Task List stored-text display regression coverage
- Existing destructive-action safety correction for Delete Task confirmation
- Existing status text readability correction so user-facing text displays "In Progress" while backend/status values remain InProgress
- Security review documentation for existing MVP implementation
- Future security hardening backlog documentation

Out of scope:

- New MVP features
- New workflows
- Time-specific due dates or due times
- UI polish beyond basic clarity and access
- Major UX redesign
- Performance testing
- Multi-user scenarios
- Future enhancements as implemented MVP features
- Subtasks

- Archiving/restoring
- Reactivation of completed tasks
- Categories/tags
- Notifications/reminders
- AI features
- Payments
- Advanced prioritization
- Navbar/logout/session guard behavior
- Completed-task restore or undo behavior
- CAPTCHA, rate limiting, lockouts, account verification, or new authentication workflows
- Token/session enforcement beyond current MVP scope
- Multi-user authorization or ownership enforcement beyond current MVP scope
- Progress Log edit/delete/revision-history behavior
- Broad DRY refactoring or architecture cleanup during post-MVP validation

Security review scope boundary:

- Security review findings may be documented as known limitations or future backlog items.
- Completed validation/test-hardening items may be documented as implemented MVP validation coverage.
- Security backlog items should not be represented as implemented MVP behavior unless they were explicitly selected, implemented, tested, and validated.
- Production-grade authentication and authorization are outside current MVP scope.
- Token/session enforcement, multi-user ownership checks, rate limiting, lockouts, CAPTCHA, account verification, logout/session guard behavior, and new authentication workflows remain future hardening items unless project scope changes.

3. Test Approach

Testing is performed using seven layers:

1. Manual Testing

Focus:

- Full workflows in the running application
- UI + API behavior
- Confirmation that implemented MVP behavior works end-to-end
- Confirmation that existing-screen clarity/access corrections work in the browser
- Confirmation that visible validation messages display clearly in existing forms
- Confirmation that Practical UX Review findings are either resolved as small existing-screen corrections or documented as known limitations

Manual testing verifies that the user can complete core TaskAnchor workflows without relying only on automated tests.

2. Backend Logic Testing (xUnit)

Focus:

- Overdue logic
- Sorting logic

- Status transitions
- LastUpdatedDate updates
- Completed task behavior
- Progress Log persistence behavior
- Progress Log validation and normalization behavior
- Task API validation behavior
- Authentication API validation behavior
- Authentication suspicious-input handling
- Password hashing behavior
- Duplicate Register email handling
- Register invalid email format handling

Backend tests verify that core system rules remain correct, that derived behavior is not stored incorrectly, that invalid data is rejected where validation hardening was selected, and that authentication inputs do not bypass expected validation or password hashing behavior.

Latest backend xUnit run:

- Date: 05/14/2026
- Command: dotnet test
- Result: 45 of 45 tests passed
- Failed: 0
- Skipped: 0
- Build: succeeded
- Notes: 2 existing nullable-reference warnings remain in ProgressLogTests.cs

3. Frontend Component Testing (Karma)

Focus:

- UI rendering
- Angular binding using ngModel
- Event handling
- Service calls
- State-driven UI behavior
- Refresh signaling
- Task object shape consistency
- Tested structural containers for cosmetic demo-readiness styling
- Existing-screen authentication access and error-message behavior
- Existing-field edit-mode behavior for title, description, PriorityLevel, DueDate, and NextAction
- Existing form validation behavior for required fields
- Frontend suspicious-input handling as plain text values
- Stored-text display regression coverage for existing Task List display fields
- Delete confirmation behavior for cancelled and accepted destructive actions
- User-facing status readability display

Strict rules:

- No direct DOM manipulation
- No document.querySelector usage in implementation
- State-driven validation only
- Returned/rendered frontend Task objects must use the full Task object shape

- Create payload tests may use create-payload-shaped objects because they test request payload behavior
- Cosmetic styling tests must remain limited to structure/access support for existing MVP screens only
- Authentication screen corrections must remain limited to existing Register/Login MVP behavior
- Existing-field edit tests should scope selectors inside edit mode when needed to avoid accidental matches from child components
- Create Task validation must remain limited to required MVP fields and existing form behavior
- Authentication validation must remain limited to existing Login/Register forms and must not expand into new authentication workflows
- Status value behavior must remain aligned with MVP status values: Active, InProgress, Completed
- User-facing status display may use "In Progress" for readability without changing stored/backend values
- Stored-text display tests must verify rendering as text without requiring new sanitization workflows or new display components

Latest Angular Karma run:

- Date: 05/14/2026
- Command: ng test
- Result: 159 of 159 tests passed
- Failed: 0
- Browser: Chrome on Windows development environment

4. Integration and E2E Testing (Playwright)

Focus:

- UI to API interaction

- Full browser flows
- Navigation behavior
- Page load behavior
- Existing MVP workflows only
- Browser dialog handling for existing destructive Delete action
- Real browser confirmation that visible validation and status text display correctly

Key integration validations:

- refreshTasks() propagation
- UI updates after API success
- No manual shared-state mutation
- Login to Task List navigation
- Register route access
- Task creation flow
- Status update flow
- Progress Log flow
- Delete confirmation cancel/accept flow

Latest Playwright run:

- Date: 05/05/2026
- Command: npx playwright test
- Result: 36 passed, 0 skipped, 0 failed
- Browsers: Chromium, Firefox, and WebKit projects from Playwright configuration

5. Manual MVP Workflow Testing

Manual MVP Workflow Testing verifies that implemented MVP behavior works end-to-end in the running application.

This is separate from automated test coverage and is used to confirm real user flow behavior.

Manual workflow testing is limited to existing MVP behavior only.

Manual MVP Workflow Testing is complete for the confirmed checklist items in this document.

6. Practical UX Review

Practical UX Review verifies clarity, access, and usability of existing MVP screens.

This is not new feature development.

Allowed focus areas:

- Clear labels
- Understandable task flow
- Practical structure and access for existing MVP screens
- Visible task status
- Visible NextAction
- Visible Progress Log entries
- Basic form readability

- Visible validation messages
- Demo-ready usability
- Cosmetic styling for existing MVP screens only when it supports clarity and demo-readiness
- Visible error messages for existing Register/Login failures
- Navigation links between existing Register and Login screens
- Small existing-screen safety corrections such as Delete Task confirmation

Not included:

- New workflows
- New features
- Major redesign
- Visual polish beyond basic clarity and access
- Navbar/logout/session guard behavior
- Completed-task reactivation, undo, restore, or archive behavior
- Progress Log edit/delete/revision-history behavior
- Broad refactoring or DRY cleanup during post-MVP validation

7. Security Review and Future Hardening Backlog

Security review verifies that security-related observations from study and source review are documented accurately without expanding the current MVP.

Security review focus:

- Injection review
- Broken Access Control review

- Existing authentication validation behavior
- Existing task/progress-log validation behavior
- Suspicious-input handling
- Stored-text display regression testing
- Password hashing behavior
- Identification of known MVP security limitations
- Future hardening backlog documentation

Security review does not mean all identified future-hardening items have been implemented.

Security review must distinguish:

- Implemented and tested MVP behavior
- Implemented validation/test-hardening items selected during Week 7
- Known limitations
- Future backlog work

4. Test Environment

The system is tested in a local development environment.

- Frontend: Angular (TaskAnchor.UI)
- Backend: ASP.NET Core API

- Database: SQL Server
- ORM: Entity Framework Core

Tools:

- Swagger
- xUnit
- Angular Karma
- Playwright
- Manual browser testing
- Source/code review
- OWASP-based security study

5. Features to Test

- Register (UI + API)
- Login (UI + API)
- Register route access
- Login/Register navigation links
- Register success navigation back to Login
- Register duplicate-email error display
- Login invalid-credential error display
- Login/Register blank-field validation

- Backend Login blank input validation
- Login/Register email length validation
- Login/Register password length validation
- Register duplicate-email validation
- Register invalid email format validation
- Login/Register suspicious-input handling
- Navigation to /tasks
- Create task
- Create Task Title validation
- Backend Create Task Title validation
- Backend Create Task text length validation
- Create Task Description textarea
- Edit task (UI + API + refresh)
- Delete task (UI + API + refresh)
- Delete Task confirmation
- View active tasks
- Status transitions (Active → InProgress → Completed)
- User-facing status text displays "In Progress" for InProgress
- Completed task terminal behavior
- Completed task exclusion from active list
- DueDate behavior
- DueDate date-only create/edit behavior
- PriorityLevel display and edit behavior
- Task sorting

- Add Progress Log entry
- Progress Log EntryText trim behavior
- Progress Log EntryText length validation
- Blank Progress Log save behavior
- View Progress Logs
- NextAction display
- NextAction edit-mode behavior
- Refresh signaling behavior
- Practical UX of existing MVP flows
- Manual workflow completion for implemented MVP behavior
- Cosmetic demo-readiness styling for existing MVP screens only
- MVP security validation already implemented for authentication suspicious-input handling
- Direct API suspicious task input handling as plain text
- Stored-text display behavior for Task List Title, Description, NextAction, and Progress Log text
- Security review documentation and future hardening backlog documentation

6. Test Cases

TC-01: Register User (API + UI)

Expected:

- User created when email and password are valid

- Input captured correctly
- Registration request sent to API when fields are valid
- Blank/whitespace email or password does not call registration service in the frontend
- Blank/whitespace email or password displays visible validation feedback
- Successful registration returns user to Login
- Duplicate registration is rejected with visible error feedback
- Backend duplicate registration returns `BadRequest("Email is already registered.")` and does not save a second user
- Register email over 255 characters returns `BadRequest("Email must be 255 characters or fewer.")` and does not save a user
- Register password over 128 characters returns `BadRequest("Password must be 128 characters or fewer.")` and does not save a user
- Clearly invalid Register email format returns `BadRequest("Email format is invalid.")` and does not save a user
- Script-like Register email input returns `BadRequest("Email format is invalid.")` and does not save a user
- SQL-like Register password input is handled as plain password input and stored only as a hash when paired with a valid email
- Password is hashed in backend registration behavior and raw password text is not stored
- Register route is accessible at `/register`
- Register page loads in Playwright

TC-02: Login User

Expected:

- Valid login succeeds
- Login request sent to API when fields are valid

- Blank/whitespace email or password does not call login service in the frontend
- Blank/whitespace email or password displays visible validation feedback
- Backend blank/whitespace Login email or password returns BadRequest("Email and password are required.")
- Login email over 255 characters returns BadRequest("Email must be 255 characters or fewer.")
- Login password over 128 characters returns BadRequest("Password must be 128 characters or fewer.")
- User navigates to /tasks after success
- Invalid login is rejected with visible error feedback
- Script-like or SQL-like Login input is handled as plain text input and does not bypass authentication
- Login → Task List navigation is covered by Playwright using a unique registered test account

TC-03: Create Task

Expected:

- Task created when required fields are valid
- Blank/whitespace Title does not call createTask
- Blank/whitespace Title does not call refreshTasks
- Blank/whitespace Title displays visible validation feedback
- Backend blank/whitespace Title returns BadRequest("Title is required.") and does not save a task
- Backend Title over 100 characters returns BadRequest("Title must be 100 characters or fewer.") and does not save a task
- Backend Description over 500 characters returns BadRequest("Description must be 500 characters or fewer.") and does not save a task

- Backend NextAction over 200 characters returns BadRequest("Next Action must be 200 characters or fewer.") and does not save a task
- Create Task Description is captured through a textarea
- Description is included in the create payload
- API called for valid create
- refreshTasks triggered after successful create
- Task visible after refresh
- Script-like and SQL-like Create Task input values are treated as plain text
- Form fields reset after successful create:
 - title
 - description
 - priorityLevel
 - dueDate
 - nextAction
- Task form validation message clears after successful create
- Playwright verifies blank-title validation
- Playwright verifies a valid task can be created through the browser and appears in the Task List with title, description, and NextAction

TC-04: Edit Task

Expected:

- Edit mode activated
- Inline editing works
- Title can be edited in edit mode

- Description can be edited in edit mode
- PriorityLevel can be edited in edit mode
- DueDate can be edited in edit mode as a date-only value
- NextAction can be edited in edit mode
- Read-only DueDate display is hidden during edit mode
- Read-only NextAction display is hidden during edit mode
- Save triggers updateTask
- refreshTasks triggered
- Changes persist after refresh
- Manual browser verification confirms DueDate edit/save persists after browser refresh

TC-05: Delete Task

Expected:

- Delete button is visible
- Clicking Delete shows a browser confirmation
- Confirmation message includes the clicked task title
- Confirmation message uses this format:
 - Delete task "Task Title"? This action cannot be undone.
- Cancelling Delete does not call deleteTask
- Cancelling Delete does not call refreshTasks
- Accepting Delete calls deleteTask(taskId)
- Accepting Delete triggers refreshTasks after delete succeeds
- Task is removed after accepted delete and refresh

- Playwright verifies cancel keeps the task visible
- Playwright verifies accept removes the task

TC-06: Status Transition

Expected:

- Active transitions to InProgress
- InProgress transitions to Completed after confirmation
- Completion confirmation can be cancelled without calling updateTaskStatus
- Completed is terminal
- updateTaskStatus called only after valid status action
- refreshTasks triggered after accepted status update
- User-facing status label displays "In Progress" for InProgress
- User-facing status button action displays "Click → In Progress" for Active tasks
- Backend/status value remains InProgress
- Playwright verifies Active → InProgress browser flow with user-facing "In Progress" text

TC-07: Overdue Logic

Expected:

- A task is overdue when:
 - Status is Active or InProgress
 - DueDate exists
 - DueDate is before the current date

- DueDate is treated as a date-only MVP field, not a date/time deadline
- Time-specific due dates, such as "due at 5:00 PM," are outside MVP scope
- Completed tasks are not overdue
- Overdue is derived, not stored

TC-08: Sorting

Expected:

- Tasks sort by:
 1. Overdue tasks
 2. Due date ascending
 3. No due date → priority High → Low

TC-09: Progress Log Add

Expected:

- Progress Log input displays for selected task
- Input captures text
- addProgressLog(taskId, text) called when text is not blank
- Backend trims EntryText before saving
- Backend saves script-like Progress Log EntryText as plain text
- Backend EntryText over 500 characters returns BadRequest("Entry Text must be 500 characters or fewer.") and does not save the entry
- refreshTasks triggered after successful Progress Log save
- Input clears after successful save

- Progress Log mode exits after successful save
- Blank/whitespace Progress Log text does not call addProgressLog
- Blank/whitespace Progress Log text does not call refreshTasks
- Blank/whitespace Progress Log save clears the input and exits Progress Log mode
- Playwright verifies Add Progress Log opens the input, saves an entry, and displays the entry on the task

TC-10: Progress Log Display

Expected:

- Progress Logs display per task
- task.progressLogs[] is rendered
- log.text or log.entryText displays correctly
- Latest or existing Progress Log entry is visible when present
- Script-like Progress Log display text renders as plain text and does not create a script element

TC-11: NextAction

Expected:

- NextAction is captured during task creation
- NextAction displays when present
- Empty NextAction does not break rendering
- NextAction can be edited through existing task edit mode
- Edit-mode NextAction field uses Angular binding and participates in the existing save flow
- Backend Create Task rejects NextAction over 200 characters

- Script-like NextAction display text renders as plain text and does not create a script element
- Playwright valid task creation verifies NextAction display after create

TC-12: Refresh Signaling

Expected:

- Task list reloads after:
 - create
 - edit
 - delete
 - status update
 - progress log
- UI updates use refreshTasks()
- Components do not manually mutate shared state

TC-13: Completed Task Behavior

Expected:

- Completed tasks are excluded from active task list
- Completed tasks do not reactivate
- Completed status remains terminal
- User receives confirmation before moving a task to Completed
- Cancelling completion keeps the task active and does not call updateTaskStatus
- Accepting completion updates the task to Completed and refreshes the active list

TC-14: Validation

Expected:

- Duplicate registration rejected
- Invalid login rejected
- Blank Login/Register fields rejected in frontend without service calls
- Backend Login blank input rejected with BadRequest
- Blank Create Task Title rejected in frontend without createTask or refreshTasks calls
- Backend Create Task blank Title rejected with BadRequest
- Backend selected text length validation rejects oversized task, Progress Log, Register, and Login input values
- Backend Register invalid email format rejected with BadRequest
- Duplicate registration shows visible frontend error message
- Invalid login shows visible frontend error message
- Required-field validation messages use visible red error styling
- Suspicious/script-like Login input is handled as plain text and does not bypass backend authentication
- Script-like Register email input is rejected as invalid email format
- SQL-like Register password input is stored only as a hash when Register email is valid
- Registration stores hashed passwords and does not store raw password text
- Playwright verifies Login blank-field validation
- Playwright verifies Register blank-field validation
- Playwright verifies Create Task blank-title validation

TC-15: UI Rendering

Expected:

- Login page loads
- Register page is routed through /register
- Register page is accessible from Login page
- Login page is accessible from Register page
- Task List page loads
- Inputs and buttons are visible
- Empty task list message displays when no active tasks exist
- Existing MVP screens use tested structural containers for demo-readiness styling
- Create Task form renders Description as a textarea
- Create Task form renders DueDate as a date input
- Status displays as "In Progress" for user readability
- Script-like Task List Title, Description, NextAction, and Progress Log text render as text and do not create script elements

TC-16: Task Object Shape Consistency

Expected:

- Returned/rendered frontend Task objects use the full Task shape:
 - taskId
 - title
 - description
 - status

- priorityLevel
- dueDate
- nextAction
- lastUpdatedDate
- Optional progressLogs may be included:
 - progressLogs?: ProgressLog[]
- Frontend ProgressLog objects may use:
 - text
 - entryText
- Backend-only userId is not required in frontend Task fixtures
- Create payload tests may remain create-payload shaped

TC-17: Manual MVP Workflow Testing

Expected:

- Register user
- Log in
- Navigate to Task List
- Create task with title, description, priorityLevel, dueDate, and nextAction
- Confirm created task appears in Task List
- Confirm Description displays when task data includes description
- Confirm Due Date displays when task data includes dueDate
- Edit task title
- Edit task description

- Edit task PriorityLevel
- Edit task DueDate
- Confirm DueDate edit/save persists after browser refresh
- Edit task NextAction
- Confirm edited task refreshes and displays updated values
- Change status from Active to InProgress
- Change status from InProgress to Completed
- Confirm completed task is removed from active list
- Add Progress Log entry to a task
- Confirm Progress Log entry displays after refresh
- Confirm blank Progress Log save closes/clears input and does not create an empty entry
- Delete task
- Confirm Delete confirmation appears before destructive delete
- Confirm deleted task is removed after accepted delete and refresh
- Confirm blank Login/Register fields show visible validation feedback
- Confirm blank Create Task Title shows visible validation feedback

TC-18: Practical UX Review

Expected:

- Login flow is understandable
- Register flow is understandable
- Task List is readable
- Existing MVP screen structure supports task follow-through

- Existing MVP screen access is understandable
- Create Task form labels are clear enough for demo use
- Create Task required-title validation is understandable
- Login/Register required-field validation is understandable
- Status change interaction is understandable
- User-facing InProgress status displays as "In Progress"
- NextAction display is visible when present
- Progress Log entry flow is understandable
- Delete action is visible and protected by confirmation
- Empty task list message is clear
- Error messages are visible for failed Register/Login attempts and required-field validation
- Remaining Practical UX Review findings are documented as known limitations if not corrected

TC-19: Cosmetic Demo-Readiness Styling

Expected:

- Login screen renders inside tested .login-card structure
- Register screen renders inside tested .register-card structure
- Create Task form renders inside tested .task-form-card structure
- Task List content renders inside tested .task-list-card structure
- Styling remains cosmetic and does not add new workflows or features
- Existing MVP controls remain visible and usable
- Login, Register, Create Task, and Task List visual styling remains consistent enough for demo use
- Angular Karma regression passes after styling updates

- Error messages use visible red styling consistent with existing Login/Register error display

TC-20: Existing Authentication Access Corrections

Expected:

- /register route uses RegisterComponent
- Login page renders a Register link
- Register page renders a Login link
- Successful registration navigates back to Login
- Duplicate registration displays backend-provided error text when available
- Failed login displays a clear Login failed message
- Blank Login/Register fields display a clear required-field message
- Angular tests cover these behaviors
- Playwright validates Register route access and Login → Task List navigation

TC-21: Authentication Suspicious-Input Handling

Expected:

- Frontend Login passes script-like or SQL-like input as plain strings to AuthService
- Frontend Register passes script-like or SQL-like input as plain strings to AuthService
- Backend Login rejects suspicious invalid credentials and does not allow bypass
- Backend Register rejects script-like email input as invalid email format
- Backend Register stores SQL-like password input as a hash when email is valid
- Backend Register does not store the raw password text

TC-22: Playwright Existing MVP Flow Coverage

Expected:

- Register page loads
- Login page loads
- Task List page loads
- Register blank-field validation displays visible error feedback
- Login blank-field validation displays visible error feedback
- Login → Task List navigation works after successful authentication
- Create Task blank-title validation displays visible error feedback
- Valid Create Task flow creates and displays a task
- Status update flow changes Active to InProgress and displays "In Progress"
- Progress Log entry can be added and displayed
- Delete confirmation cancel keeps the task visible
- Delete confirmation accept removes the task
- Full Playwright suite passes across configured browser projects with 0 skipped tests

TC-23: Security Review and Future Hardening Documentation

Expected:

- Injection review findings are documented accurately.
- Broken Access Control review findings are documented accurately.
- Implemented MVP security validations are separated from future hardening items.

- Completed validation/test-hardening backlog items are identified as implemented only when validated.
- No production-grade security claims are made.
- Known security limitations are documented as limitations or future backlog items.
- Future security backlog items do not expand current MVP scope.
- Security backlog items include injection-related hardening and broken-access-control hardening.
- Current capstone documentation remains focused on implemented MVP behavior, validation results, known limitations, and future work.

TC-24: Stored Text Display Regression Coverage

Expected:

- Script-like Task Title text displays as text in the Task List.
- Script-like Task Title text does not create a script element.
- Script-like Task Description text displays as text in the Task List.
- Script-like Task Description text does not create a script element.
- Script-like NextAction text displays as text in the Task List.
- Script-like NextAction text does not create a script element.
- Script-like Progress Log text displays as text in the Task List.
- Script-like Progress Log text does not create a script element.
- Existing Task List display behavior remains unchanged.
- No production behavior change is required if Angular interpolation remains safe.

7. Current Automated Test Coverage

Backend:

- Rule coverage for overdue behavior
- Rule coverage for sorting behavior
- Rule coverage for status transitions
- API behavior coverage for implemented backend flows
- Authentication validation behavior for duplicate registration and invalid login
- Authentication suspicious-input handling
- Password hashing behavior during registration
- Backend Create Task blank Title validation
- Backend Create Task Title length validation
- Backend Create Task Description length validation
- Backend Create Task NextAction length validation
- Backend Progress Log EntryText trim behavior
- Backend Progress Log script-like plain-text behavior
- Backend Progress Log EntryText length validation
- Backend suspicious task input handling as plain text
- Backend Register email length validation
- Backend Register password length validation
- Backend Login email length validation
- Backend Login password length validation
- Backend Login blank input validation
- Backend valid Register regression coverage

- Backend valid Login regression coverage
- Backend Register invalid email format validation
- Backend script-like Register email rejection coverage

Latest backend xUnit run:

- Date: 05/14/2026
- Command: dotnet test
- Result: 45 of 45 tests passed
- Failed: 0
- Skipped: 0
- Build: succeeded
- Notes: 2 existing nullable-reference warnings remain in ProgressLogTests.cs

Backend Week 7 tests added or updated:

- CreateTask_WithBlankTitle_ReturnsBadRequest_And_DoesNotSaveTask
- CreateProgressLogEntry_TrimsEntryText_BeforeSaving
- CreateProgressLogEntry_WithScriptLikeText_SavesAsPlainText
- CreateTask_WithSuspiciousText_SavesValuesAsPlainText
- CreateTask_WithTitleOverMaxLength_ReturnsBadRequest_And_DoesNotSaveTask
- CreateTask_WithDescriptionOverMaxLength_ReturnsBadRequest_And_DoesNotSaveTask
- CreateTask_WithNextActionOverMaxLength_ReturnsBadRequest_And_DoesNotSaveTask
- CreateProgressLogEntry_WithEntryTextOverMaxLength_ReturnsBadRequest_And_DoesNotSaveEntry
- Register_WithEmailOverMaxLength_ReturnsBadRequest_And_DoesNotSaveUser

- Register_WithPasswordOverMaxLength_ReturnsBadRequest_And_DoesNotSaveUser
- Login_WithEmailOverMaxLength_ReturnsBadRequest
- Login_WithPasswordOverMaxLength_ReturnsBadRequest
- Login_WithBlankEmailOrPassword_ReturnsBadRequest
- Register_WithDuplicateEmail_ReturnsBadRequest_And_DoesNotSaveSecondUser
- Register_WithValidEmailAndPassword_ReturnsOk_And_SavesUser
- Login_WithValidCredentials_ReturnsOk
- Register_WithClearlyInvalidEmail_ReturnsBadRequest_And_DoesNotSaveUser
- Register_WithScriptLikeEmail_ReturnsBadRequest_And_DoesNotSaveUser
- Register_With_SqlLike_Password_Input_Stores_Email_And_Hashes_Password

Frontend:

- RegisterComponent
- LoginComponent
- TaskListComponent
- TaskFormComponent
- AuthService
- TaskService
- Routing spec
- AppComponent

Frontend coverage includes:

- UI rendering
- Angular binding behavior

- Service interaction
- Status transitions
- User-facing status display as "In Progress"
- Edit/save flow
- Delete behavior
- Delete confirmation cancel/accept behavior
- Progress Log full flow
- Refresh signaling
- Task object shape normalization
- Tested structural containers for Login, Register, Create Task, and Task List demo-readiness styling
- Register route exposure
- Login/Register navigation links
- Register success navigation back to Login
- Register failed/duplicate-email error display
- Login failed-attempt error display
- Login/Register blank-field validation
- Login/Register suspicious-input handling as plain text
- Completion confirmation before moving tasks to Completed
- Scoped edit-mode tests for title, description, PriorityLevel, DueDate, and NextAction
- Priority display and edit-mode support
- DueDate date-only create/edit support
- NextAction edit-mode input rendering and binding through the existing edit/save flow
- Create Task Description textarea
- Create Task Description payload support

- Create Task field reset behavior after successful create
- Create Task required Title validation
- Create Task validation error styling
- Blank Progress Log save guard and close/clear behavior
- Stored-text display regression coverage for script-like Task Title
- Stored-text display regression coverage for script-like Task Description
- Stored-text display regression coverage for script-like NextAction
- Stored-text display regression coverage for script-like Progress Log text

Latest frontend Karma run:

- Date: 05/14/2026
- Command: ng test
- Result: 159 of 159 tests passed
- Failed: 0
- Browser: Chrome on Windows development environment

Frontend Week 7 tests added:

- should render script-like task title as text without creating a script element
- should render script-like task description as text without creating a script element
- should render script-like next action as text without creating a script element
- should render script-like progress log text without creating a script element

E2E:

- Login page loads

- Register page loads
- Task List page loads
- Register blank-field validation
- Login blank-field validation
- Login → Task List navigation
- Create Task blank-title validation
- Valid Create Task flow
- Status update flow
- Progress Log add/display flow
- Delete confirmation cancel flow
- Delete confirmation accept flow

Latest Playwright run:

- Date: 05/05/2026
- Command: npx playwright test
- Result: 36 passed, 0 skipped, 0 failed
- Notes:
 - Existing skipped Register page test was enabled and corrected to /register.
 - Playwright coverage is limited to existing MVP flows only.
 - Unique test data and explicit waits are used where needed because local task data can grow during repeated E2E runs.

Security validation coverage currently implemented:

- Frontend suspicious-input tests for Login
- Frontend suspicious-input tests for Register

- Backend suspicious-input test for Login
- Backend SQL-like password hashing test for Register
- Backend script-like Register email rejection test
- Password hashing behavior confirmed by backend test coverage
- Blank Login/Register validation blocks frontend service calls
- Blank backend Login input returns BadRequest
- Blank Create Task Title validation blocks frontend createTask and refreshTasks calls
- Blank backend Create Task Title returns BadRequest
- Duplicate Register email returns BadRequest and does not save a second user
- Register invalid email format returns BadRequest and does not save a user
- Backend selected length-limit validations cover task fields, Progress Log EntryText, Register email/password, and Login email/password
- Direct API suspicious task input coverage confirms script-like and SQL-like task input values save as plain text
- Stored-text display regression coverage confirms script-like Title, Description, NextAction, and Progress Log values render as text without creating script elements

Security validation not implemented in MVP:

- Token/session enforcement
- Multi-user authorization and ownership enforcement
- Rate limiting
- Lockouts
- CAPTCHA
- Account verification
- Logout/session guard behavior
- CSRF protection for future cookie/session authentication

- Full production dependency vulnerability remediation process
- Full production-grade authorization architecture

8. Manual MVP Workflow Checklist

- [x] Register user
- [x] Log in
- [x] Navigate to Task List
- [x] Create task with title, priorityLevel, dueDate, and nextAction
- [x] Confirm created task appears in Task List
- [x] Confirm Description displays with task data
- [x] Confirm Due Date displays with task data
- [x] Edit task title
- [x] Confirm edited task refreshes and displays updated title
- [x] Edit task DueDate
- [x] Confirm DueDate edit/save persists after browser refresh
- [x] Change status from Active to InProgress
- [x] Change status from InProgress to Completed
- [x] Confirm completed task is removed from active list
- [x] Add Progress Log entry to a task
- [x] Confirm Progress Log entry displays after refresh
- [x] Delete task

- [x] Confirm deleted task is removed after refresh
- [x] Confirm Delete confirmation appears before destructive delete

Additional authentication/access manual confirmations:

- [x] Confirm Login page provides access to Register
- [x] Confirm Register page provides access back to Login
- [x] Confirm successful registration returns to Login
- [x] Confirm duplicate registration displays visible error message
- [x] Confirm invalid login displays visible error message
- [x] Confirm blank Login fields display required-field message and block login call
- [x] Confirm blank Register fields display required-field message and block register call

Additional form/manual validation confirmations:

- [x] Confirm editing NextAction after task creation updates and displays correctly after save/refresh
- [x] Confirm editing PriorityLevel persists after save
- [x] Confirm blank Progress Log save clears/closes input and does not create an empty entry
- [x] Confirm Create Task blank Title displays dark-red validation message
- [x] Confirm Create Task valid submission clears the Title validation message
- [x] Confirm Description display with task data
- [x] Confirm Due Date display with task data
- [x] Confirm DueDate edit/save after browser refresh

Manual browser verification still needed:

- None currently identified for core MVP workflow completion.

9. Practical UX Review Checklist

- [x] Confirm Login flow is understandable
- [x] Confirm Register flow is understandable
- [x] Confirm Task List is readable
- [x] Confirm existing MVP screen structure supports task follow-through
- [x] Confirm existing MVP screen access is understandable
- [x] Confirm Create Task form labels are clear enough for demo use
- [x] Confirm Create Task validation is understandable
- [x] Confirm Login/Register validation is understandable
- [x] Confirm Status change interaction is understandable
- [x] Confirm NextAction display is visible when present
- [x] Confirm Progress Log entry flow is understandable
- [x] Confirm Delete action is visible and predictable after Delete confirmation correction
- [x] Confirm empty task list message is clear

Practical UX Review status:

Full existing-screen Practical UX Review was confirmed on 05/05/2026.

Screens reviewed:

- Login

- Register
- Task List
- Create Task
- Edit Task mode
- Progress Log flow
- validation messages

Practical UX Review observation:

Initial manual screen review found that existing MVP screens are functional and core information is visible, including task status, priority, NextAction, Progress Log entries, and task actions. However, the initial layout and visual structure were plain and crowded.

A cosmetic styling/demo-readiness pass was completed for existing MVP screens only. Login, Register, Create Task, and Task List now use tested structural card containers with consistent black, white, and gold styling. The styling work remained cosmetic and did not add new MVP features or workflows.

Additional Practical UX Review/access observations:

- Login originally had no visible Register access.
- Register was implemented but not routed.
- Register success did not return the user to Login.
- Invalid Login attempts displayed no visible message.
- Duplicate Register attempts displayed no visible message even though the backend returned a duplicate-email error.
- Login/Register navigation links originally had inconsistent styling.
- These issues were corrected as clarity/access fixes for existing MVP authentication screens.
- Login/Register blank-field validation was added as an existing-form clarity/access correction.

- Create Task Title validation was added as an existing-form clarity/access correction.
- Create Task Description was changed to a textarea to better match the existing field purpose.

Practical UX Review Findings from Task List Screen Review:

- NextAction display label clarity was addressed.
 - Original problem: NextAction text appeared as plain text, which made it harder to distinguish from description or Progress Log content.
 - Resolution: NextAction displays with a clear label.
 - Scope: Completed as a clarity/access fix for an existing MVP screen.
- Progress Log display label clarity was addressed.
 - Original problem: Progress Log text appeared as plain text, which made it harder to distinguish from NextAction or description content.
 - Resolution: Progress Log entries display with a clear label.
 - Scope: Completed as a clarity/access fix for an existing MVP screen.
- NextAction edit-mode gap was addressed.
 - Original problem: NextAction was captured on create and displayed in the task list, but edit mode did not expose an edit field for the existing NextAction value.
 - Resolution: Added edit-mode NextAction input using ngModel and normalized edit-mode tests.
 - Scope: Completed as an existing-field edit gap, not a new feature.
- PriorityLevel edit-mode gap was addressed.
 - Original problem: PriorityLevel displayed as an existing Task field but was not editable in edit mode.

- Resolution: Added edit-mode PriorityLevel select.
 - Scope: Completed as an existing-field edit gap, not a new feature.
-
- DueDate edit-mode gap was addressed.
 - Original problem: DueDate displayed as an existing Task field but was not editable in edit mode.
 - Resolution: Added edit-mode date-only DueDate input.
 - Scope: Completed as an existing-field edit gap, not a new feature.
-
- Delete Task confirmation gap was addressed.
 - Original problem: Delete Task happened immediately after clicking Delete, with no confirmation warning.
 - Resolution: Added title-specific browser confirmation before deleting.
 - Scope: Completed as a safety/clarity correction for an existing destructive action.
 - Confirmation text: Delete task "Task Title"? This action cannot be undone.
-
- Status readability gap was addressed.
 - Original problem: User-facing status text displayed "InProgress" as one word.
 - Resolution: User-facing status text now displays "In Progress."
 - Scope: Completed as a readability correction for existing MVP status display.
 - Backend/status value remains InProgress.

Remaining Practical UX Review findings / known limitations:

- Add Progress Log button remains visible/clickable after Add Progress Log mode is already open, but clicking it again does not appear to do anything.
- Edit mode and Add Progress Log mode can be open at the same time on the same task.

- A task can be deleted while Edit mode and/or Add Progress Log mode are open, although Delete now requires confirmation.
- Multiple interaction modes may allow unsaved text to be lost if the user switches actions before saving.
- Create Task labels do not use colons, while Task List detail labels use colons; this creates minor label-format inconsistency.
- Create Task card and Task List card styling are similar enough that the sections could be easier to distinguish visually.
- Create Task heading and Task List heading appear at different sizes.
- Date inputs in Create Task and Edit Task may appear in a different font than surrounding form fields depending on browser rendering.
- Code has repeated patterns and could benefit from DRY cleanup later.

Current decision:

- The Delete confirmation and status readability issues were corrected because they were small, high-value existing-screen clarity/safety fixes.
- Remaining interaction-mode findings are documented as known limitations.
- Do not expand MVP scope into edit/progress-log mode redesign unless explicitly needed later.
- Do not start broad DRY refactoring during post-MVP validation unless required by a defect.

10. E2E Coverage Gaps

Current Playwright coverage:

- Login page loads
- Register page loads

- Task List page loads
- Register blank-field validation
- Login blank-field validation
- Login → Task List navigation
- Create Task blank-title validation
- Valid Create Task flow
- Status update flow
- Progress Log flow
- Delete confirmation cancel flow
- Delete confirmation accept flow

Latest Playwright run:

- Date: 05/05/2026
- Command: npx playwright test
- Result: 36 passed, 0 skipped, 0 failed

Previously documented E2E coverage gaps and current status:

- Register page load test should be unskipped/revalidated now that /register exists
 - Status: Complete
- Login → TaskList navigation test
 - Status: Complete
- Task creation flow test
 - Status: Complete
- Status update flow test

- Status: Complete
- Progress Log flow test
 - Status: Complete
- Login/Register blank validation flow tests if Playwright is expanded
 - Status: Complete
- Create Task validation flow test if Playwright is expanded
 - Status: Complete
- Delete confirmation flow after safety correction
 - Status: Complete

Remaining E2E coverage gaps:

- No required Playwright gaps remain from the previously documented existing-MVP E2E gap list.

E2E expansion must stay limited to existing MVP flows only.

11. Risks and Problem Areas

- Test object shape inconsistency
- Frontend/backend contract mismatches
- Refresh pattern misuse
- Overdue must remain derived
- DueDate must remain date-only in the MVP; due times are a future enhancement

- Risk of Angular overengineering
- Limited authentication enforcement
- Authentication validation must remain simple and within MVP scope
- Suspicious-input validation tests do not replace full production security hardening
- Integration issues not caught in unit tests
- Practical UX issues may block demo clarity even if automated tests pass
- Manual workflow issues may reveal behavior gaps not caught by unit tests
- Cosmetic styling must remain limited to existing MVP screens and must not become feature expansion
- Login/Register access and error feedback must remain clarity/access corrections, not expanded authentication/session management
- Completing a task is terminal and removes it from the active list; accidental completion risk was resolved with confirmation before Completed
- Delete is destructive; accidental delete risk was reduced with title-specific confirmation before Delete
- Edit-mode selector tests can accidentally match child create-form controls if not scoped to edit mode
- Create Task validation should not add new task workflows or unsupported required fields
- Backend validation hardening should remain focused and test-driven
- Register invalid email validation should not be expanded into a new authentication workflow
- Stored-text display regression testing should not become a broad UI rewrite
- Playwright tests create local task records; repeated local runs can grow task data and require unique test data and explicit waits
- Browser-specific timing may require stable waits in E2E tests
- Vite/Angular error overlays can block Playwright clicks if the app is not restarted after a template compile issue
- Remaining interaction-mode overlap findings are known limitations unless explicitly selected for future correction

- Broad DRY cleanup could introduce risk during post-MVP validation and should not be started unless it is required by a defect
- MVP-level access control is intentionally limited
- Token/session enforcement is outside MVP scope
- Multi-user record ownership enforcement is outside MVP scope
- Direct API hardening should be future work if the project is expanded beyond local MVP/capstone use
- Future security backlog items should not be confused with implemented MVP test coverage
- Production deployment would require additional security hardening beyond current MVP validation

12. Known Issues / Required Fixes

Backend Test Build Blocker - Resolved

A backend test build blocker occurred because `Microsoft.AspNetCore.OpenApi` was using version `10.0.5` while the API targets `net9.0`.

Resolution:

- Changed `Microsoft.AspNetCore.OpenApi` from `10.0.5` to `9.0.0`
- Re-ran `dotnet test`
- Result: 25 of 25 backend tests passed

Latest backend test update:

- Backend xUnit passed 45 of 45 tests on 05/14/2026 after Week 7 backend validation and authentication hardening tests were added.

Remaining note:

- Backend test run completed with 2 nullable-reference warnings in `ProgressLogTests.cs`.

Test Integrity

Task object shape normalization has been completed for checked frontend returned/rendered Task tests.

Required returned/rendered frontend Task structure:

- taskId
- title
- description
- status
- priorityLevel
- dueDate
- nextAction
- lastUpdatedDate

Optional:

- progressLogs?: ProgressLog[]

Frontend ProgressLog test/display shape:

- text?
- entryText?

Create payload tests should not be forced into the full Task shape.

Valid create payload shape:

- title
- description
- priorityLevel
- dueDate
- nextAction

Latest test integrity update:

- Edit-mode title, description, PriorityLevel, DueDate, and NextAction tests were normalized to scope selectors inside ``.edit-task-fields`` where needed.
- This avoids accidental matches from the rendered `<app-task-form>` child component.
- TaskForm create tests include Description textarea behavior, reset behavior, required Title validation, and task-form error styling.
- Authentication form tests include blank-field validation and suspicious-input string handling.
- Delete tests include cancelled and accepted confirmation behavior.
- Status display tests verify that InProgress displays to the user as "In Progress."
- Task List stored-text display regression tests confirm script-like Title, Description, NextAction, and Progress Log values render as text without script elements.
- Angular Karma passed after latest validation updates: 159 of 159 tests passed.

E2E Coverage

Playwright coverage has been expanded and remains limited to existing MVP flows only.

Completed E2E coverage:

- Login page loads
- Register page loads
- Task List page loads
- Register blank-field validation
- Login blank-field validation
- Login → Task List navigation
- Create Task blank-title validation
- Valid Create Task flow
- Status update flow
- Progress Log flow
- Delete confirmation cancel flow
- Delete confirmation accept flow

Latest Playwright verification:

- Date: 05/05/2026
- Command: npx playwright test
- Result: 36 passed, 0 skipped, 0 failed

Required future E2E targets:

- None currently required from the documented E2E gap list.

- Any future Playwright additions must remain limited to existing MVP flows only.

Manual Validation

Manual MVP Workflow Testing is complete for the currently confirmed checklist items. Browser verification confirmed Login, Register, Task List, Create Task, Edit Task, status transitions, Completed task exclusion, NextAction display/edit behavior, PriorityLevel edit behavior, DueDate edit/save after refresh, Description display, Due Date display, Progress Log save/display after refresh, Delete Task, deleted task removal after refresh, Delete confirmation, and refresh behavior.

Additional browser verification confirmed:

- Login page provides access to Register
- Register page provides access back to Login
- Register success returns to Login
- Duplicate Register attempt displays a visible error message
- Invalid Login attempt displays a visible error message
- Login/Register link styling is visually consistent
- Editing NextAction after task creation updates and displays correctly after save/refresh
- Editing PriorityLevel persists after save
- Editing DueDate persists after save and browser refresh
- Description displays with task data
- Due Date displays with task data
- Blank Progress Log save clears/closes input and does not create an empty entry
- Create Task blank Title displays a dark-red validation message
- Create Task valid submission clears the Title validation message
- Login blank-field validation displays a red required-field message and blocks submission

- Register blank-field validation displays a red required-field message and blocks submission

Manual browser verification still needed:

- None currently identified for core MVP workflow completion.

Practical UX Review:

- Full existing-screen Practical UX Review was confirmed on 05/05/2026.
- Findings are documented in Section 9.
- Delete confirmation and status readability were corrected as small existing-screen clarity/safety fixes.
- Remaining interaction-mode and styling consistency observations are known limitations.

Cosmetic Demo-Readiness Styling

Cosmetic demo-readiness styling has been applied to existing MVP screens only.

Completed styling work:

- Login screen wrapped in tested .login-card structure
- Register screen wrapped in tested .register-card structure
- Create Task form wrapped in tested .task-form-card structure
- Task List content wrapped in tested .task-list-card structure
- Consistent black, white, and gold styling applied
- Font consistency applied across Login, Register, Create Task, and Task List cards
- Login/Register link styling aligned
- Required-field error styling uses dark red consistent with Login/Register error display

- Angular Karma regression passed after latest validation/styling updates: 159 of 159 tests passed

This work did not add new MVP features, new workflows, priority-edit behavior beyond existing field edit support, redesign, or scope expansion.

Authentication Access/Error Feedback Corrections

Resolved:

- /register route was missing.
- Login page did not expose a Register link.
- Register page did not expose a Login link.
- Register did not navigate back to Login after successful registration.
- Invalid Login attempt displayed no visible message.
- Duplicate Register attempt displayed no visible message.
- Login/Register link styling was inconsistent.
- Blank Login/Register fields called authentication services without frontend validation.

Resolution:

- Added /register route to existing RegisterComponent.
- Added tested Register link on Login page.
- Added tested Login link on Register page.
- Added tested Register success navigation back to Login.
- Added tested Login failure message.
- Added tested Register failure/duplicate-email message.

- Displayed backend-provided duplicate-email text from `HttpErrorResponse.error`.
- Aligned Login/Register link styling.
- Added tested blank-field validation for Login and Register.
- Added frontend suspicious-input tests for Login and Register.
- Added backend suspicious-input authentication tests.
- Added Playwright coverage for Register route, Register validation, Login validation, and Login → Task List navigation.

Completed Status Accidental Click Risk - Resolved

Observation:

- InProgress tasks remain in the active list.
- Completed tasks are excluded from the active list.
- Completed is terminal and cannot be reactivated in the MVP.
- User identified that accidentally completing a task could remove it from the active list.

Resolution:

- Added a confirmation prompt before moving a task to Completed.
- Cancelling the confirmation keeps the task active and does not call `updateTaskStatus`.
- Accepting the confirmation updates the task to Completed and triggers `refreshTasks`.
- Active → InProgress behavior remains unchanged.
- InProgress → Completed behavior remains valid only after confirmation.

Scope boundary maintained:

- Did not add undo, restore, reactivation, archive, completed-task list, or a new workflow.

- Completed-task reactivation remains out of scope.

Verification:

- Added frontend test coverage for cancelled completion confirmation.
- Added frontend test coverage for accepted completion confirmation.
- Updated existing InProgress → Completed test to confirm before completion.
- Angular Karma passed after confirmation updates.
- Manual browser verification confirmed Cancel keeps the task active and OK completes the task and removes it from the active list.

Delete Task Accidental Click Risk - Resolved

Observation:

- Delete Task originally happened immediately after clicking Delete.
- User identified that a destructive action should allow the user to confirm the correct task before deletion.

Resolution:

- Added title-specific browser confirmation before Delete Task.
- Confirmation message includes the clicked task title:
 - Delete task "Task Title"? This action cannot be undone.
- Cancelling Delete does not call deleteTask.
- Cancelling Delete does not call refreshTasks.
- Accepting Delete calls deleteTask(taskId).
- Accepting Delete triggers refreshTasks after delete succeeds.

Scope boundary maintained:

- Did not add archive, restore, undo, completed-task list, or a new deletion workflow.
- Treated confirmation as an existing-screen safety/clarity correction.

Verification:

- Added Angular coverage for cancelled delete confirmation.
- Added Angular coverage for accepted delete confirmation.
- Updated existing delete tests to accept window.confirm before expecting delete behavior.
- Added Playwright coverage for Delete confirmation cancel flow.
- Added Playwright coverage for Delete confirmation accept flow.
- Angular Karma passed 159 of 159.
- Playwright passed 36 of 36.

Status Text Readability - Resolved

Observation:

- User-facing task status displayed "InProgress" as one word.
- This reflected the internal status value but was less readable in the UI.

Resolution:

- User-facing current status text now displays "In Progress."
- User-facing status action text now displays "Click → In Progress."
- Backend/status values remain unchanged as InProgress.

- Status transition behavior remains unchanged.

Scope boundary maintained:

- Did not rename backend enum values.
- Did not change API contracts.
- Did not add a new status.
- Treated this as a display/readability correction only.

Verification:

- Added/updated Angular coverage for ``getStatusLabel('InProgress')`` and ``getStatusLabel(1)`` displaying "In Progress."
- Updated status button text expectations.
- Updated Playwright status update test expectations.
- Angular Karma passed 159 of 159.
- Playwright passed 36 of 36.

NextAction Edit-Mode Gap - Resolved

Observation:

- NextAction was captured during task creation and displayed in the task list.
- Edit mode included title and description fields, but did not expose an edit field for NextAction.

Resolution:

- Added edit-mode NextAction input.
- Bound edit-mode NextAction through the existing task object using ngModel.

- Kept the existing save flow through `updateTask` and `refreshTasks`.
- Normalized edit-mode tests so selectors are scoped inside the edit-mode container.

Scope boundary maintained:

- Did not add a new workflow.
- Did not add a new field.
- Did not add advanced editing behavior.
- Treated `NextAction` as an existing MVP field.

Verification:

- Added frontend test coverage for edit-mode `NextAction` input rendering.
- Normalized related edit-mode title and description tests.
- Angular Karma passed after `NextAction` update.
- Manual browser verification confirmed `NextAction` edit/save behavior.

PriorityLevel Edit-Mode Gap - Resolved

Observation:

- `PriorityLevel` existed as an MVP Task field and was used in create/sorting behavior, but the Task List initially did not display or edit it clearly.

Resolution:

- Added read-only Priority display.
- Used `getPriorityLabel(task.priorityLevel)` so numeric backend values display as Low, Medium, or High.

- Added edit-mode PriorityLevel select.
- Kept the existing save flow through updateTask and refreshTasks.

Scope boundary maintained:

- Treated PriorityLevel as an existing MVP field.
- Did not add advanced prioritization.

Verification:

- Added Angular coverage for Priority display and edit-mode Priority select.
- Manual browser verification confirmed Priority edit/save persisted.

DueDate Edit-Mode Gap - Resolved

Observation:

- DueDate existed as an MVP Task field and was used in sorting/overdue behavior, but edit mode did not expose DueDate editing.

Resolution:

- Added edit-mode DueDate input.
- Used input type="date".
- Displayed date-only value using substring(0, 10).
- Used [value] and (input) instead of transformed two-way ngModel to avoid Angular unsupported two-way action binding.
- Hid read-only DueDate while editing.
- Kept DueDate as a date-only MVP field.

Scope boundary maintained:

- Did not add due times.
- Did not change DueDate to DateTime deadline behavior.

Verification:

- Added Angular coverage for edit-mode DueDate input, date-only value formatting, and hiding read-only DueDate while editing.
- Manual browser verification confirmed DueDate edit/save persists after browser refresh.

Progress Log Blank Save Behavior - Resolved

Observation:

- Pressing Save Progress Log with blank or whitespace text originally attempted to save or did nothing visibly.

Resolution:

- Blank/whitespace Progress Log save does not call addProgressLog.
- Blank/whitespace Progress Log save does not call refreshTasks.
- Blank/whitespace Progress Log save clears the input and exits Progress Log mode.

Scope boundary maintained:

- Did not add Progress Log edit/delete/revision history.
- Did not add a new Progress Log workflow.

Verification:

- Added Angular coverage for blank/whitespace Progress Log save behavior.
- Manual browser verification confirmed the input clears/closes and no empty entry is created.
- Playwright verifies Progress Log entry add/display behavior for nonblank entry.

Create Task Form Validation and Description Textarea - Resolved

Observation:

- Create Task Description was better suited to a textarea than a single-line input.
- Create Task Title could be blank/whitespace in the frontend before validation was added.
- Backend Create Task Title also needed direct validation because frontend validation can be bypassed.

Resolution:

- Changed Create Task Description input to textarea.
- Confirmed Description is included in create payload.
- Added reset coverage for Description, PriorityLevel, DueDate, and NextAction after successful create.
- Confirmed DueDate create input is type="date".
- Added required Title validation.
- Blank/whitespace Title does not call createTask.
- Blank/whitespace Title does not call refreshTasks.
- Existing form values remain in place for correction.
- Added visible task-form error message.
- Styled task-form error in dark red to match Login/Register validation/error styling.

- Valid create clears the validation message.
- Added backend Create Task validation so blank/whitespace Title returns BadRequest("Title is required.") and does not save a task.

Scope boundary maintained:

- Did not add new task fields.
- Did not make optional MVP fields required.
- Did not add status to the Create Task form because status is backend/default create behavior.

Verification:

- Angular Karma passed after Create Task validation work.
- Backend xUnit passed after backend Title validation work.
- Manual browser verification confirmed Title validation message appears in dark red and clears after successful valid task creation.
- Playwright verifies Create Task blank-title validation.
- Playwright verifies valid Create Task browser flow.

Backend Text Length Validation - Resolved for Selected Fields

Observation:

- Several existing text fields did not have backend maximum length guards.
- Very large values could cause avoidable storage bloat, poor UI rendering, or unnecessary API/database load.
- The selected fields were existing MVP inputs, not new functionality.

Resolution:

- Added backend Create Task Title length validation: 100 characters.
- Added backend Create Task Description length validation: 500 characters.
- Added backend Create Task NextAction length validation: 200 characters.
- Added backend Progress Log EntryText length validation: 500 characters.

Scope boundary maintained:

- Did not add new task fields.
- Did not add new workflows.
- Did not add frontend-only validation as a substitute for backend validation.
- Authentication input length limits were handled under authentication validation hardening.

Verification:

- Added backend xUnit coverage for oversized Title rejection.
- Added backend xUnit coverage for oversized Description rejection.
- Added backend xUnit coverage for oversized NextAction rejection.
- Added backend xUnit coverage for oversized Progress Log EntryText rejection.
- Backend xUnit passed 35/35 after the selected task/progress-log text field limits were added.
- Latest backend xUnit now passes 45/45 after later authentication validation additions.

Backend Progress Log Input Normalization - Resolved

Observation:

- Progress Log EntryText already had blank-input handling and trim behavior, but explicit backend test confirmation was needed.
- Script-like Progress Log text also needed confirmation as plain text input behavior.

Resolution:

- Confirmed backend trims EntryText before saving.
- Confirmed script-like EntryText is stored as plain text.
- Added length validation for EntryText over 500 characters.

Scope boundary maintained:

- Did not add Progress Log edit/delete/revision history.
- Did not add a new Progress Log workflow.

Verification:

- Added backend test: CreateProgressLogEntry_TrimsEntryText_BeforeSaving.
- Added backend test: CreateProgressLogEntry_WithScriptLikeText_SavesAsPlainText.
- Added backend test:
CreateProgressLogEntry_WithEntryTextOverMaxLength_ReturnsBadRequest_And_DoesNotSaveEntry.
- Backend xUnit passed after each TDD/regression step.
- Latest backend xUnit passes 45/45.

Authentication Suspicious-Input Handling - Resolved for MVP Validation

Observation:

- User requested verification that Login/Register input cannot be used for injection-style behavior.

Resolution:

- Added frontend Login test proving script-like and SQL-like input is passed as plain text to AuthService.login.
- Added frontend Register test proving script-like and SQL-like input is passed as plain text to AuthService.register.
- Added backend Login test proving suspicious/script-like input does not bypass authentication.
- Added backend Register test proving SQL-like password input is hashed and not stored raw when paired with a valid email.
- Added backend Register test proving script-like email input is rejected as invalid email format.

Scope boundary maintained:

- Did not add CAPTCHA.
- Did not add rate limiting.
- Did not add lockouts.
- Did not add account verification.
- Did not add token/session guard behavior.
- Did not add a new authentication workflow.

Verification:

- Angular Karma passed.
- Backend xUnit passed 45 of 45.
- Manual browser verification confirmed Login/Register blank-field validation displays red messages and blocks blank submission.

Authentication Input Length and Invalid Email Validation - Resolved

Observation:

- Register/Login email and password inputs needed focused backend length validation.
- Backend Login blank input behavior needed parity with frontend validation.
- Duplicate Register email behavior needed regression coverage.
- Register invalid email format validation was selected as a small existing-behavior validation hardening item.

Resolution:

- Register email values over 255 characters return BadRequest("Email must be 255 characters or fewer.") and are not saved.
- Register password values over 128 characters return BadRequest("Password must be 128 characters or fewer.") and are not saved.
- Login email values over 255 characters return BadRequest("Email must be 255 characters or fewer.").
- Login password values over 128 characters return BadRequest("Password must be 128 characters or fewer.").
- Login blank/whitespace email or password returns BadRequest("Email and password are required.").
- Duplicate Register email returns BadRequest("Email is already registered.") and does not save a second user.
- Valid Register behavior has regression coverage.
- Valid Login behavior has regression coverage.
- Clearly invalid Register email format returns BadRequest("Email format is invalid.") and does not save a user.
- Script-like Register email input returns BadRequest("Email format is invalid.") and does not save a user.
- SQL-like Register password input remains plain password input and is stored only as a hash when paired with a valid email.

Scope boundary maintained:

- Did not add account verification.
- Did not add CAPTCHA.
- Did not add rate limiting.
- Did not add account lockout.
- Did not add session/token enforcement.
- Did not add a new authentication workflow.

Verification:

- Added backend xUnit tests for Register/Login input length validation.
- Added backend xUnit test for Login blank input validation.
- Added backend xUnit duplicate Register email regression test.
- Added backend xUnit valid Register regression test.
- Added backend xUnit valid Login regression test.
- Added backend xUnit invalid Register email format test.
- Added backend xUnit script-like Register email rejection test.
- Updated backend suspicious Register password test to use a valid email with SQL-like password input.
- Latest backend xUnit passed 45 of 45.

Direct API Suspicious Task Input Coverage - Resolved

Observation:

- Suspicious authentication input was already tested, but task input fields also needed direct API coverage to confirm script-like and SQL-like values are treated as plain text data.

Resolution:

- Added backend xUnit coverage for Create Task with script-like and SQL-like values.
- Confirmed suspicious-looking task values are saved as plain text data.

Scope boundary maintained:

- Did not add new task validation behavior beyond the selected validation hardening.
- Did not add a new workflow.
- Did not claim this replaces production security hardening.

Verification:

- Added backend test: `CreateTask_WithSuspiciousText_SavesValuesAsPlainText`.
- Backend xUnit passed after the test.
- Latest backend xUnit passes 45/45.

Stored Text Display Regression Coverage - Resolved

Observation:

- Angular interpolation lowers stored-XSS risk as long as unsafe dynamic HTML patterns are not introduced.
- Existing Task List display fields should have regression coverage confirming script-like values render as text and do not create script elements.

Resolution:

- Added frontend regression test for script-like task Title display.
- Added frontend regression test for script-like task Description display.

- Added frontend regression test for script-like NextAction display.
- Added frontend regression test for script-like Progress Log display.

Scope boundary maintained:

- Did not add new UI behavior.
- Did not add new sanitization workflow.
- Did not add new display component.
- Did not change production code because existing Angular interpolation behavior was sufficient.

Verification:

- Added test: should render script-like task title as text without creating a script element.
- Added test: should render script-like task description as text without creating a script element.
- Added test: should render script-like next action as text without creating a script element.
- Added test: should render script-like progress log text without creating a script element.
- Angular Karma passed 159 of 159.

DueDate Time-Specific Deadline Boundary

Observation:

- A possible enhancement was identified: using due dates with specific times, such as homework due at 5:00 PM.

Decision:

- DueDate remains a date-only MVP field.
- Time-specific due dates or due times are outside MVP scope.

- This prevents expansion into DateTime deadline behavior during post-MVP validation.

Scope boundary maintained:

- No application code changes were made for time-specific due dates.
- No test changes were made to require due-time behavior.
- Overdue remains derived from the date-only DueDate behavior.

Security Review and Future Hardening Backlog - Documented

Observation:

- A security review was performed using OWASP A05:2025 Injection and OWASP A01:2025 Broken Access Control as study references.
- The purpose was to identify whether the current MVP has obvious vulnerabilities and to create future backlog cards for security hardening.

Current assessment from reviewed project code and documentation:

- No clear SQL injection or command injection vulnerability was identified in the reviewed code snippets.
- Reviewed backend behavior appears to use Entity Framework Core/LINQ rather than string-concatenated dynamic SQL.
- No raw SQL, command execution, stored procedure execution, or dynamic command construction was identified in the reviewed snippets.
- Frontend and backend suspicious-input authentication tests exist for MVP validation.
- Script-like and SQL-like Login input is handled as plain text and does not bypass backend authentication.
- Registration password input is hashed and raw password text is not stored.
- Script-like Register email input is rejected as invalid email format after selected validation hardening.

- Direct API suspicious task input coverage confirms task fields are treated as plain text.
- Stored-text display regression coverage confirms script-like existing Task List fields render as text without script elements.
- Angular text interpolation lowers stored-XSS risk as long as unsafe dynamic HTML patterns are not introduced.
- The larger security limitation is access control hardening, not confirmed injection behavior.
- Current MVP uses basic authentication and simplified single-user behavior.
- Token/session enforcement and multi-user ownership enforcement are outside MVP scope.

Security backlog cards created from Injection review:

1. Add backend Title validation for Create Task.
 - Status: Complete.
2. Add backend length limits for text fields.
 - Status: Complete for selected task/progress-log text fields.
3. Add backend Progress Log input normalization review.
 - Status: Complete.
4. Add stored XSS regression tests for task display fields.
 - Status: Complete.
5. Search codebase for unsafe dynamic execution patterns.
 - Status: Active / future review.
6. Add direct API tests for suspicious task input.
 - Status: Complete.
7. Add backend authorization/session hardening as future enhancement.
 - Status: Future.
8. Add duplicate and invalid email validation review.

- Status: Complete.
9. Document security limitations in README or Test Plan.
 - Status: In progress through documentation alignment.
 10. Add dependency vulnerability review before final release.
 - Status: Active / future review.

Security backlog cards created from Broken Access Control review:

1. Add backend authentication enforcement for Task APIs.
2. Replace hardcoded UserId behavior with authenticated user ownership.
3. Add ownership checks to Progress Log endpoints.
4. Add direct API tests for force browsing protection.
5. Add frontend unauthorized-response handling.
6. Add CSRF review for future browser-based authentication.
7. Review CORS configuration before deployment.
8. Add deny-by-default authorization policy.
9. Add access-control logging for rejected protected requests.
10. Add rate limiting for authentication and protected APIs.
11. Add secure session/token lifetime review.
12. Document Broken Access Control MVP limitations.

Additional future security review items:

- Review file upload exposure before adding attachments, imports, profile images, or task files.
- Review path traversal risk before adding file, export, import, download, or generated-file features.

- Review unsafe deserialization risk before adding import, backup restore, bulk API, or complex serialized data features.

Scope boundary maintained:

- These cards are future backlog items unless explicitly selected later.
- The security review did not add new MVP features or workflows.
- Token/session enforcement remains outside current MVP scope.
- Multi-user ownership enforcement remains outside current MVP scope.
- CAPTCHA, rate limiting, lockouts, account verification, and new authentication workflows remain outside current MVP scope.
- Navbar/logout/session guard behavior remains outside current MVP scope.
- Current documentation should not claim production-grade security.

Current decision:

- Security review findings should be documented as completed validation hardening, future hardening backlog, and known limitations as appropriate.
- Security hardening should not interrupt final capstone documentation, demo preparation, poster preparation, presentation preparation, or work log completion unless a small existing-behavior defect is explicitly selected.

13. Security Review Summary

Security review date:

- 05/06/2026 through Week 7 validation updates on 05/14/2026

Security study areas:

- OWASP A05:2025 Injection
- OWASP A01:2025 Broken Access Control

Implemented MVP security validation:

- Login rejects invalid credentials.
- Duplicate registration is rejected.
- Passwords are hashed during registration.
- Blank Login/Register fields are blocked in the frontend before service calls.
- Backend Login rejects blank/whitespace email or password input.
- Register/Login email and password length limits are enforced in the backend.
- Register invalid email format is rejected in the backend.
- Script-like Register email input is rejected as invalid email format.
- Suspicious/script-like Login input is passed as plain text on the frontend and does not bypass backend authentication.
- Suspicious/script-like Register input is passed as plain text on the frontend.
- SQL-like registration password input is hashed and not stored raw when paired with a valid email.
- Create Task blank Title is blocked in the frontend before createTask and refreshTasks calls.
- Create Task blank Title is rejected by the backend.
- Create Task selected text fields have backend length limits.
- Progress Log EntryText is trimmed and length-limited in the backend.
- Script-like Progress Log EntryText is stored as plain text.
- Direct API suspicious task input is covered and stored as plain text.
- Existing Task List Title, Description, NextAction, and Progress Log display fields have stored-text display regression coverage.

Security limitations intentionally outside MVP scope:

- Token/session enforcement.
- Logout/session guard behavior.
- Multi-user authorization.
- Per-user task ownership enforcement.
- Completed production authorization model.
- CAPTCHA.
- Rate limiting.
- Lockouts.
- Account verification.
- CSRF protection for future browser-session authentication.
- Production-grade dependency vulnerability remediation process.
- Production deployment hardening.

Current security conclusion:

- No confirmed injection vulnerability was found in the reviewed code snippets.
- The main security limitation is MVP-level access control/authentication enforcement.
- Selected validation and regression hardening items were completed for existing MVP behavior.
- Remaining security findings are documented as future backlog items and should not be treated as implemented features.
- If TaskAnchor is expanded beyond capstone/local MVP use, access control and authorization hardening should be prioritized before production use.

14. Changelog

v1.16 (05/14/2026)

- Updated document version to v1.16 and Last Updated date to 05/14/2026.
- Updated current work log status to 70.32 hours worked and 29.68 hours remaining toward the 100-hour capstone requirement.
- Updated latest backend xUnit result to 45 of 45 tests passed.
- Updated latest Angular Karma result to 159 of 159 tests passed.
- Confirmed latest Playwright result remains 36 passed, 0 skipped, 0 failed.
- Added completed Week 7 backend validation-hardening coverage:
 - Backend Create Task blank Title validation.
 - Backend Create Task Title length validation.
 - Backend Create Task Description length validation.
 - Backend Create Task NextAction length validation.
 - Backend Progress Log EntryText trim behavior.
 - Backend Progress Log script-like plain-text behavior.
 - Backend Progress Log EntryText length validation.
 - Direct API suspicious task input plain-text handling.
- Added completed Week 7 authentication validation-hardening coverage:
 - Register email length validation.
 - Register password length validation.
 - Login email length validation.
 - Login password length validation.
 - Backend Login blank input validation.

- Duplicate Register email regression coverage.
- Valid Register regression coverage.
- Valid Login regression coverage.
- Register invalid email format validation.
- Script-like Register email rejection coverage.
- SQL-like Register password hashing coverage with valid email.
- Added completed Week 7 stored-text display regression coverage:
 - Script-like task Title renders as text without creating a script element.
 - Script-like task Description renders as text without creating a script element.
 - Script-like NextAction renders as text without creating a script element.
 - Script-like Progress Log text renders as text without creating a script element.
- Added TC-24: Stored Text Display Regression Coverage.
- Updated TC-01 Register User with duplicate email, length limit, invalid email format, script-like email rejection, and SQL-like password hashing expectations.
- Updated TC-02 Login User with backend blank input and length-limit expectations.
- Updated TC-03 Create Task with backend blank Title and text length-limit expectations.
- Updated TC-09 Progress Log Add with trim, script-like plain-text, and EntryText length-limit expectations.
- Updated TC-10 Progress Log Display with script-like display regression expectation.
- Updated TC-11 NextAction with length-limit and script-like display regression expectations.
- Updated TC-14 Validation to include backend selected validation-hardening coverage.
- Updated TC-15 UI Rendering to include stored-text display regression behavior.
- Updated TC-16 Task Object Shape Consistency to include frontend ProgressLog text/entryText shape.
- Updated TC-21 Authentication Suspicious-Input Handling to reflect current Register invalid-email behavior.

- Updated Current Automated Test Coverage with backend Week 7 test names.
- Updated Current Automated Test Coverage with frontend Week 7 stored-text display regression test names.
- Added Backend Text Length Validation resolved section.
- Added Backend Progress Log Input Normalization resolved section.
- Added Authentication Input Length and Invalid Email Validation resolved section.
- Added Direct API Suspicious Task Input Coverage resolved section.
- Added Stored Text Display Regression Coverage resolved section.
- Updated Security Review and Future Hardening Backlog section with completed/in-progress/future status for relevant security backlog cards.
- Added file upload exposure, path traversal, and unsafe deserialization/import review as future security review items.
- Maintained scope boundaries: no new MVP features, no new workflows, no token/session enforcement, no multi-user ownership enforcement, no CAPTCHA, no rate limiting, no lockouts, no account verification, no logout/session guard behavior, no CSRF implementation, no role-based access control, no Progress Log edit/delete/revision history, no due times, and no broad refactoring.

v1.15 (05/06/2026)

- Updated document version to v1.15 and Last Updated date to 05/06/2026.
- Updated current work log status to 66.48 hours worked and 33.52 hours remaining toward the 100-hour capstone requirement.
- Updated Purpose section to include security review documentation, known-limitation documentation, and future hardening backlog documentation.
- Updated Scope section to include security review documentation for existing MVP implementation and future security hardening backlog documentation.
- Added security review scope boundary clarifying that future hardening items are not implemented MVP behavior.
- Updated Test Approach from six layers to seven layers by adding Security Review and Future Hardening Backlog.

- Updated Test Environment to include source/code review and OWASP-based security study.
- Updated Features to Test to include MVP security validation already implemented for authentication suspicious-input handling and future hardening backlog documentation.
- Added TC-23: Security Review and Future Hardening Documentation.
- Updated Current Automated Test Coverage to distinguish implemented MVP security validation from security items not implemented in MVP.
- Updated Risks and Problem Areas to include MVP-level access control limits, token/session enforcement limits, multi-user ownership limits, backend validation parity review, and production security hardening boundaries.
- Added Security Review and Future Hardening Backlog section under Known Issues / Required Fixes.
- Added Section 13: Security Review Summary.
- Documented security review date as 05/06/2026.
- Documented OWASP A05:2025 Injection and OWASP A01:2025 Broken Access Control as study areas.
- Documented current security assessment: no clear SQL injection or command injection vulnerability identified in reviewed snippets.
- Documented that reviewed backend behavior appears to use Entity Framework Core/LINQ rather than string-concatenated dynamic SQL.
- Documented that suspicious authentication input is already tested and does not bypass backend authentication.
- Documented that password hashing is covered by MVP backend tests.
- Documented that the main security limitation is MVP-level access control/authentication enforcement.
- Documented security backlog cards from Injection review:
 - Add backend Title validation for Create Task.
 - Add backend length limits for text fields.
 - Add backend Progress Log input normalization review.
 - Add stored XSS regression tests for task display fields.

- Search codebase for unsafe dynamic execution patterns.
- Add direct API tests for suspicious task input.
- Add backend authorization/session hardening as future enhancement.
- Add duplicate and invalid email validation review.
- Document security limitations in README or Test Plan.
- Add dependency vulnerability review before final release.
- Documented security backlog cards from Broken Access Control review:
 - Add backend authentication enforcement for Task APIs.
 - Replace hardcoded UserId behavior with authenticated user ownership.
 - Add ownership checks to Progress Log endpoints.
 - Add direct API tests for force browsing protection.
 - Add frontend unauthorized-response handling.
 - Add CSRF review for future browser-based authentication.
 - Review CORS configuration before deployment.
 - Add deny-by-default authorization policy.
 - Add access-control logging for rejected protected requests.
 - Add rate limiting for authentication and protected APIs.
 - Add secure session/token lifetime review.
 - Document Broken Access Control MVP limitations.
- Maintained scope boundaries: no token/session enforcement, no multi-user ownership enforcement, no CAPTCHA, no rate limiting, no lockouts, no account verification, no new auth workflows, no logout/session guard behavior, and no production-grade security claim added.
- Clarified that security backlog work should not interrupt final capstone documentation, demo preparation, poster preparation, presentation preparation, or work log completion unless a small existing-behavior defect is explicitly selected.

v1.14 (05/05/2026)

- Updated document version to v1.14 and Last Updated date to 05/05/2026.
- Updated latest Angular Karma result to 155 of 155 tests passed using ng test.
- Updated latest Playwright result to 36 passed, 0 skipped, and 0 failed using npx playwright test.
- Documented that previously listed Playwright E2E gaps are complete.
- Documented Register page load Playwright test as enabled and corrected to /register.
- Documented Register blank-field validation Playwright coverage.
- Documented Login blank-field validation as included in full Playwright coverage.
- Documented Login → Task List Playwright coverage using unique registered test account.
- Documented Create Task blank-title validation Playwright coverage.
- Documented valid Create Task Playwright coverage for title, description, and NextAction display.
- Documented Status update Playwright coverage for Active → InProgress with user-facing "In Progress" display.
- Documented Progress Log Playwright coverage for add/display behavior.
- Documented Delete confirmation Playwright coverage for cancel and accept flows.
- Added TC-22 for Playwright Existing MVP Flow Coverage.
- Updated TC-05 Delete Task to require title-specific confirmation before destructive delete.
- Added Delete Task Accidental Click Risk as resolved.
- Documented Delete confirmation message: Delete task "Task Title"? This action cannot be undone.
- Documented Cancel path does not call deleteTask or refreshTasks.
- Documented Confirm path calls deleteTask(taskId) and refreshTasks after delete succeeds.
- Updated TC-06 Status Transition to document user-facing "In Progress" display while preserving InProgress backend/status values.
- Added Status Text Readability as resolved.

- Documented that user-facing status text and status action text now display "In Progress."
- Confirmed backend/status values remain unchanged as InProgress.
- Updated Manual MVP Workflow Checklist to mark DueDate edit/save after browser refresh confirmed.
- Updated Manual MVP Workflow Checklist to mark Description display with task data confirmed.
- Updated Manual MVP Workflow Checklist to mark Due Date display with task data confirmed.
- Marked full Practical UX Review checklist complete after manual existing-screen review.
- Documented Practical UX Review findings from 05/05/2026.
- Documented remaining Practical UX Review known limitations:
 - Add Progress Log button remains clickable while Add Progress Log mode is already open.
 - Edit mode and Add Progress Log mode can be open at the same time.
 - Task can be deleted while Edit mode and/or Add Progress Log mode are open, although Delete now requires confirmation.
 - Unsaved text may be lost when switching interaction modes.
 - Create Task label colon formatting differs from Task List detail labels.
 - Create Task and Task List card styling could be more visually distinct.
 - Create Task and Task List headings appear at different sizes.
 - Date inputs may render with browser-specific font differences.
 - Code could benefit from future DRY cleanup.
- Documented decision to correct only small existing-screen safety/readability issues now and leave broader interaction-mode/design/DRY cleanup as known limitations.
- Updated E2E Coverage Gaps section to show no required Playwright gaps remain from the previously documented list.
- Updated risks to include growing local Playwright-created task data, browser timing, Vite overlay interference after template compile issues, interaction-mode limitations, and DRY cleanup risk.

- Maintained scope boundaries: no new MVP features, no new workflows, no completed-task restore/reactivation/archive behavior, no due times, no Progress Log edit/delete/revision history, and no broad refactoring.

v1.13 (05/04/2026)

- Updated latest Angular Karma result to 151 of 151 tests passed using ng test.
- Updated latest backend xUnit result to 27 of 27 tests passed using dotnet test.
- Added Create Task Description textarea coverage and documented Description as included in the create payload.
- Added Create Task field reset coverage for Description, PriorityLevel, DueDate, and NextAction after successful create.
- Confirmed DueDate create input is type="date".
- Added required Title validation for Create Task.
- Documented that blank/whitespace Create Task Title does not call createTask or refreshTasks.
- Added visible dark-red task-form validation error styling aligned with Login/Register error styling.
- Manually verified Create Task Title validation appears in dark red and clears after successful valid create.
- Added Login blank-field validation coverage and implementation.
- Added Register blank-field validation coverage and implementation.
- Manually verified Login/Register required-field messages display in red and block blank submission.
- Added frontend suspicious-input tests for Login and Register.
- Added backend Login suspicious-input test confirming script/SQL-like input does not bypass authentication.
- Added backend Register suspicious-input test confirming email is stored as trimmed text and password is hashed instead of stored raw.
- Added TC-21 for Authentication Suspicious-Input Handling.

- Updated validation, frontend coverage, backend coverage, manual validation, risk, and known-issue sections for authentication validation/security handling.
- Documented PriorityLevel read-only display and edit-mode support.
- Documented DueDate edit-mode support and date-only edit input behavior.
- Documented blank Progress Log save close/clear behavior.
- Kept Practical UX Review checklist unchecked pending explicit full manual screen review confirmation.
- Kept Playwright gaps documented and limited to existing MVP flows only.

v1.12 (05/03/2026)

- Updated latest Angular Karma result to 135 of 135 tests passed using ng test.
- Clarified that DueDate remains a date-only MVP field.
- Documented time-specific due dates or due times as outside MVP scope.
- Updated overdue logic expectations to prevent scope expansion into DateTime deadline behavior.
- Added NextAction edit-mode behavior to test scope and test cases.
- Added frontend coverage note for scoped edit-mode tests covering title, description, and NextAction.
- Documented that edit-mode selector tests were normalized to avoid accidental matches from the rendered TaskFormComponent.
- Documented NextAction edit-mode gap as resolved in automated tests.
- Added manual verification note that editing NextAction after task creation still needs browser confirmation before being marked manually verified.
- Updated Practical UX Review findings to show NextAction label clarity, Progress Log label clarity, and NextAction edit-mode gap as addressed.
- Kept Practical UX Review checklist unchecked pending explicit full manual screen review confirmation.
- Manually verified that editing NextAction after task creation updates and displays correctly after save.

v1.11 (05/01/2026)

- Updated latest Angular Karma result to 120 of 120 tests passed using ng test.
- Added tested completion confirmation behavior before marking a task Completed.
- Added coverage for cancelled completion confirmation to verify updateTaskStatus is not called.
- Added coverage for accepted completion confirmation to verify updateTaskStatus and refreshTasks are called.
- Updated existing InProgress to Completed status test to account for confirmation behavior.
- Manually verified that cancelling completion keeps the task active.
- Manually verified that accepting completion completes the task and removes it from the active list.
- Resolved Completed Status Accidental Click Risk without adding undo, restore, reactivation, archive, completed-task list, or a new workflow.

v1.10 (04/30/2026)

- Updated latest Angular Karma result to 117 of 117 tests passed using ng test watch-mode.
- Added /register route coverage using red/green TDD.
- Exposed existing RegisterComponent through /register.
- Added tested Register link on Login page for users without an account.
- Added tested Login link on Register page for users who already have an account.
- Added Register success navigation back to Login using red/green TDD.
- Added tested .register-card structure for Register cosmetic demo-readiness styling.
- Applied Register black/white/gold styling aligned with Login, Create Task, and Task List.
- Added visible Login failure message using red/green TDD.
- Added visible Register failure/duplicate-email message using red/green TDD.

- Confirmed duplicate Register displays backend-provided message from `HttpErrorResponse.error`.
- Aligned Login/Register navigation link styling.
- Updated Manual MVP Workflow Checklist to mark Register user complete after manual browser verification.
- Added authentication/access manual confirmations for Login-to-Register access, Register-to-Login access, Register success return to Login, duplicate Register error display, and invalid Login error display.
- Updated TC-01 to include Register success navigation and duplicate-registration feedback.
- Updated TC-02 to include invalid-login feedback.
- Updated TC-14 to include visible frontend validation/error feedback.
- Updated TC-15 to include `/register` routing and Login/Register navigation links.
- Updated TC-18 to include Register flow and error-message clarity.
- Updated TC-19 to include `.register-card` styling.
- Added TC-20 for existing authentication access corrections.
- Updated automated frontend coverage notes to include Register routing, Register/Login links, Register success navigation, Login error display, and Register error display.
- Updated E2E notes because `/register` is now exposed and the skipped Register Playwright test should be revisited.
- Added known issue for accidental Completed status risk.
- Clarified that confirmation before Completed is allowed, but `undo/restore/reactivation/archive/completed-list` behavior remains out of scope.
- Kept Practical UX Review checklist unchecked pending explicit full manual screen review confirmation.

v1.9 (04/30/2026)

- Updated latest Angular Karma result to 111 of 111 tests passed using `ng test --watch=false`.
- Corrected frontend test count after removal of a duplicated Task List list-container render test.

- Added tested Task List structure for cosmetic demo-readiness styling using .task-list-card.
- Recorded red/green TDD flow for Task List styling support: structure test failed before wrapper, passed after wrapper.
- Added Task List cosmetic styling with black, white, and gold visual direction.
- Added font consistency across Login, Create Task, and Task List card containers.
- Documented that Login, Create Task, and Task List visual styling are now consistent.
- Added TC-19 for cosmetic demo-readiness styling coverage.
- Updated Practical UX Review observation to note that the initial plain/crowded layout was addressed through cosmetic styling for existing MVP screens only.
- Kept Practical UX Review checklist unchecked pending explicit full manual screen review confirmation.
- Updated Manual MVP Workflow Checklist to reflect confirmed browser verification for Login, Task List, Create Task, Edit Task, status transitions, Completed exclusion, NextAction display, Progress Log save/display after refresh, Delete Task, deleted task removal after refresh, and refresh behavior.
- Left Register user unchecked because that checklist item had not been separately confirmed in the current manual testing record.

v1.8 (04/29/2026)

- Recorded Manual MVP Workflow Testing confirmations for Progress Log display after refresh and Delete Task removal after refresh
- Updated latest Angular Karma result: 108 of 108 tests passed using ng test --watch=false
- Updated latest backend xUnit result: 25 of 25 tests passed using dotnet test
- Documented Manual MVP Workflow Testing as partially complete, with Practical UX Review still pending unless separately confirmed
- Added tested Login screen structure for demo-readiness styling; Angular Karma passed 109/109 after Login styling update.
- Added tested Login and Create Task structure for cosmetic demo-readiness styling; Angular Karma passed 110/110 after styling updates.

v1.7 (04/27/2026)

- Added Manual MVP Workflow Testing section and checklist
- Added Practical UX Review section and checklist for existing MVP flows
- Clarified that UX review is limited to clarity, access, usability, and structure of existing MVP screens
- Clarified that UX polish, redesign, new workflows, and new features are out of scope
- Added E2E Coverage Gaps section
- Updated test approach from four layers to six layers
- Updated known issues to reflect completed Task object shape normalization and remaining E2E/manual validation work
- Updated Playwright coverage notes to record latest run result: 3 passed and 6 skipped across Chromium, Firefox, and WebKit
- Documented skipped Register page E2E test as a routing/coverage gap because RegisterComponent is implemented and unit-tested but no /register route is currently exposed
- Marked TaskList page load Playwright coverage as started while keeping remaining E2E flows listed as gaps
- Recorded filtered Playwright run confirming 3 active E2E smoke tests passed after excluding skipped Register route coverage
- Recorded latest Angular Karma test run: 108 of 108 frontend tests passed using ng test --watch=false
- Resolved backend test build blocker by aligning Microsoft.AspNetCore.OpenApi package version with net9.0 target framework.
- Recorded latest backend xUnit test run: 25 of 25 tests passed with 2 nullable-reference warnings.

v1.6 (04/25/2026)

- Updated test coverage to reflect full frontend MVP completion
- Added edit flow testing (UI + API + refresh)

- Added full Progress Log testing (input, save, display)
- Added validation for refreshTasks across all operations
- Removed outdated gaps (task refresh, missing UI features)
- Introduced test consistency requirement for Task object shape

v1.5 (04/23/2026)

- Added refresh signaling tests
- Updated create-to-visible behavior
- Introduced ngModel binding validation

v1.4 (04/22/2026)

- Added Login UI, Task List UI, and Task Create UI to scope
- Added task creation behavior and service interaction testing
- Added login → task list navigation validation
- Documented known gap: task list does not refresh after creation
- Removed duplicate test case
- Updated frontend testing approach to include service interaction

v1.3 (04/20/2026)

- Added Integration and End-to-End Testing approach
- Added Playwright to testing environment
- Expanded scope to include full system testing (UI + API)
- Added TC-01B (Register page browser test)
- Added TC-01C (Register integration flow)

- Added TC-21 and TC-22 for Playwright-based E2E testing
- Updated risks to include integration and browser testing concerns
- Shifted testing emphasis from heavy TDD to full system validation per instructor feedback

v1.2 (04/18/2026)

- Added frontend testing approach section
- Added Register UI test case (TC-01A)
- Updated test environment to include Angular Karma
- Updated scope to reflect frontend implementation
- Updated risks to include frontend integration and complexity concerns

v1.1 (04/16/2026)

- Updated test cases to reflect implemented authentication endpoints
- Added tests for invalid login and duplicate registration handling
- Added test for Progress Log updating LastUpdatedDate
- Added test for correct task update using TaskId
- Updated test environment to reflect current implementation state